bootPLS: Bootstrap Hyperparameter Selection for PLS Models and Extensions

Frédéric Bertrand and Myriam Maumy

LIST3N – Troyes Technology University – MaMOT – Eut+ – France

Conclusion

The goal of `bootPLS` is to provide:

- easy to use functions to apply
- several non-parametric stable
- bootstrap-based techniques
- to determine the numbers of components
- in Partial Least Squares and
- sparse Partial Least Squares
- linear or generalized linear regression
- support for parallel computing

Availability

Visit `https://fbertran.github.io/bootPLS/`
As the `bootPLS` CRAN package.

Related works

`bootPLS` implements several algorithms that were published as a book chapter and two articles.

A new bootstrap-based stopping criterion in PLS component construction, J. Magnanensi, M. Maumy-Bertrand, N. Meyer and F. Bertrand (2016), in *The Multiple Facets of Partial Least Squares and Related Methods*, doi:10.1007/978-3-319-40643-5_18.

A new universal resample-stable bootstrap-based stopping criterion for PLS component construction, J. Magnanensi, F. Bertrand, M. Maumy-Bertrand and N. Meyer, (2017), *Statistics and Computing*, 27, 757–774. doi:10.1007/s11222-016-9651-4

New developments in Sparse PLS regression, J. Magnanensi, M. Maumy-Bertrand, N. Meyer and F. Bertrand, (2021), *Frontiers in Applied Mathematics and Statistics*, doi: 10.3389/fams.2021.693126.

Why another algorithm to find hyper-parameters for PLS based models?

An example

Michel Tenenhaus' reported in his book, La régression PLS (1998) Technip, Paris, that most of the expert biologists claimed that this dataset features two latent variables, which is tantamount to the PLS model having two components.

Set up 6-fold CV (K=6), 100 times (NK=2), and use random=TRUE to randomly create folds for repeated CV.

```
bbb2 <- plsRglm::cv.plsR(log(x11)~., data=pine,
nt=6, K=6, NK=100, verbose=FALSE)
```

Display the results of the cross-validation.

```
plot(plsRglm::cvtable(summary(bbb2)), type='CVQ2')
#> CV Q2 criterion:
#>   0   1
#>   0 100
```

In that PLSR setting, the Q2 criterion is recommended for complete data (without missing data). A model with 1 component is selected by the cross-validation. Hence the Q2 criterion (1 component, see Figure 1A) does not agree with the experts (2 components).

PLS (Y,T) retrieves a better number of components

The package features our PLS (Y,T) bootstrap based algorithm to select the number of components in plsR regression. It is implemented with the `nbcomp.bootplsR` function.

```
nbcomp.bootplsR(Y=ypine,X=Xpine,R=500)
```

To replicate 20 times the bootstrap analysis based on 500 bootstrap sample.

```
replicate(20,nbcomp.bootplsR(Y=ypine, X=Xpine,
R=500, verbose=FALSE, parallel="multicore",
ncpus = 2))
```

A model with two components (see Figure 2A) should be selected using our bootstrap based algorithm to select the number of components. Hence the number of component selected with our algorithm agrees with what was stated by the experts.

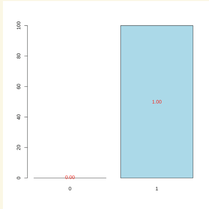

Figure 1A: Q2 CV based number of components selection

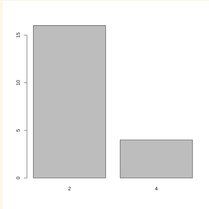

Figure 1B: bootstrap based number of components selection

Bootstrap (Y,X) for the coefficients with number of components updated for each resampling

For each of the (Y,X) bootstrapped sample, the function `coefs.plsR.adapt.ncomp` performs a (Y,T) bootstrap to fnd the relevant number of components.

```
coefs.plsR.adapt.ncomp(datasetpine,sample(1:nrow(d
atasetpine)))
#>  [1]  2.000000000  8.064227044 -0.003019447
#>  [4] -0.054389603 -0.005212285 -0.109946405
#>  [7]  0.038799785 -0.078324545 -1.334080678
#> [10] -0.045021804 -0.390730689  0.054696227
```

Parallel computing support with the `ncpus` and `parallel='multicore'` options.

```
coefs.plsR.adapt.ncomp(datasetpine,sample(1:nrow(d
atasetpine)),ncpus=2,parallel='multicore')
```

sPLS (Y,T) Bootstrap

The package also features our bootstrap based algorithm to select, for a given η value, the number of components in spls regression. It is implemented with the `nbcomp.bootspls` function.

```
nbcomp.bootspls(x=Xpine,y=ypine,eta=.5)

#> eta = 0.5
#> [1] 1
#> [1] 2
#> [1] 3
#> [1] 4
#>
#> Optimal parameters: eta = 0.5, K = 3
#> $mspemat
#>
#> eta= 0.5 , K= 3 0.9595453
#>
```

```
#> $eta.opt
#> [1] 0.5
#>
#> $K.opt
#> [1] 3
```

A `doParallel` and `foreach` based parallel computing version of the algorithm is implemented as the `nbcomp.bootspls.para` function.

```
nbcomp.bootspls.para(x=Xpine,y=ypine,eta=c(.2,.5))
#> [1] "eta = 0.2"
#> [1] 2
#> [1] 3
#> [1] "eta = 0.5"
#> [1] 2
#> [1] 3
#>
#> Optimal parameters: eta = 0.5, K = 2
```

Aze real dataset

Binary logistic plsRglm (Y,T) Bootstrap

Perform the bootstrap based algorithm with the `nbcomp.bootplsRglm` function. By default 250 resamplings are carried out.

```
nbcomp.bootplsRglm(plsRglm(y~.,data=dataset,10,mod
ele='pls-glm-family',family='binomial'))
```

Sparse PLSGLR (Y,T) Bootstrap

The package also features our bootstrap based algorithm to select, for a given η value, the number of components in sgpls regression. It is implemented in the `nbcomp.bootsgpls` function.

```
nbcomp.bootsgpls((prostate$x)[,1:30], prostate$y,
R=250, eta=0.2, typeBCa = FALSE)
#> [1] "eta = 0.2"
#> [1] "K = 1"
#> [1] "K = 2"
#> [1] "K = 3"
#> [1] "K = 4"
#> [1] "K = 5"
#> Warning: glm.fit: fitted probabilities
#> numerically 0 or 1 occurred
#>
#> Optimal parameters: eta = 0.2, K = 4
```

A `doParallel` and `foreach` based parallel computing version of the algorithm is implemented as the `nbcomp.bootspls.para` function.